

Prueba Sumativa 1

Curso: Desarrollo Web Móvil

28 de abril de 2026

Restaurant

El proyecto entregado corresponde a una aplicación cliente-servidor incompleta para la gestión de pedidos de un restaurant. El frontend está desarrollado con **HTML**, **CSS** y **JavaScript** puro, mientras que el backend está desarrollado con **Node.js** y **Express**. El backend trabaja con arreglos en memoria y no utiliza base de datos.

Instrucciones

- No se debe migrar el proyecto a TypeScript.
- No se deben agregar frameworks de frontend.
- Se debe trabajar respetando la estructura base entregada.
- Se deben completar las partes faltantes sin reescribir completamente el proyecto.
- No puede utilizar IA en ningún caso.
- Puede utilizar ThunderClient o Postman, o similares.
- Para correr backend y frontend utilice en cada carpeta:
 - `npm install`
 - `npm run dev`
- El proyecto ya se refresca solo al tener cambios
- Vaya haciendo `console.log()` para ir debuggando
- Hay un proyecto backend que ya funciona, para correrlo entre a la carpeta respectiva y ejecute:
 - `npm install`
 - `npm run start:dist`

Lo puede utilizar para trabajar en el frontend si es que no ha completado el backend aún. Este backend funcional corre en el puerto 3001. Por lo que en el archivo `api.js` del frontend debe cambiar la URL.

Parte 1: Backend

1. Completar la respuesta de POST /ingredientes (15 %)

Completar la función `crearIngrediente()`, debe rechazar lo siguiente:

- Nombre vacío
- Stock no numérico
- Stock menor que 0

Cuando rechace devuelve el código HTTP 400 y un mensaje explicativo para cada caso (los 3 casos que se señalan).

En el body de la Request viene lo siguiente:

```
{
  "nombre": "Tomate",
  "stock": 10
}
```

Una vez que lo valida, lo guarda en los datos del programa, devuelve el código 201 junto con un mensaje de éxito y el ingrediente creado en la Response.

```
{
  mensaje: "...",
  ingrediente: {
    "nombre": "Tomate",
    "stock": 10
  }
}
```

2. Completar la validación de POST /platos (25 %)

Completar la función `crearPlato()`. Actualmente la ruta permite crear platos sin validar correctamente los datos.

Se debe validar lo siguiente antes de crear el plato:

- El nombre no puede estar vacío.
- El precio debe ser un número mayor que 0.
- El campo ingredientes debe existir y ser un arreglo.
- El arreglo ingredientes no puede estar vacío.
- Cada elemento debe tener `ingredienteId`.
- Cada `ingredienteId` debe existir en el sistema.
- Cada cantidad debe ser numérica y mayor que 0.

En caso de que alguna validación falle, se debe responder con código HTTP 400 y un mensaje explicativo en formato JSON para cada caso.

El body de la Request tiene el siguiente formato:

```
{
  "nombre": "Completo",
  "precio": 3500,
  "ingredientes": [
    { "ingredienteId": 1, "cantidad": 1 },
    { "ingredienteId": 3, "cantidad": 2 }
  ]
}
```

Si la validación es correcta, se debe:

- Crear el nuevo plato.
- Guardarlo en los datos del sistema.
- Responder con código HTTP 201.
- Retornar el plato creado con su detalle.

3. Mejorar la respuesta de GET /pedidos (20 %)

Actualmente la ruta GET /pedidos retorna los pedidos con sus items en formato básico, sin suficiente detalle.

Se debe modificar la respuesta para que cada pedido incluya:

- La cantidad de items distintos (`cantidadItems`).
- El total general del pedido.
- El detalle de cada item con nombre del plato y subtotal.

El subtotal de cada item se calcula como:

`subtotal = precio del plato * cantidad`

El total del pedido corresponde a la suma de todos los subtotales.

Formato esperado de cada pedido en la respuesta:

```
{
  "id": 1,
  "fecha": "2026-04-27T10:00:00.000Z",
  "cantidadItems": 2,
  "total": 9500,
  "itemsDetalle": [
    {
      "platoId": 1,
      "nombre": "Hamburguesa",
      "cantidad": 1,
      "subtotal": 5000
    },
    {
      "platoId": 2,
      "nombre": "Cheeseburger",
      "cantidad": 1,
      "subtotal": 4500
    }
  ]
}
```

Consideraciones:

- Se debe obtener el nombre y precio del plato desde los datos existentes.
- Si un plato no existe, se puede mostrar como “Plato eliminado”.
- Todos los cálculos deben ser correctos.

Parte 2: Frontend

4. Validar el formulario de ingredientes en el frontend (10 %)

Modificar el archivo `frontend/js/ingredientes.js` para validar los datos antes de enviarlos al backend.

Al presionar el botón `Guardar ingrediente`, se debe validar:

- El nombre no puede estar vacío.
- El stock debe ser numérico.
- El stock debe ser mayor o igual a 0.

Si alguna validación falla:

- Se debe mostrar un mensaje de error en `mensajeIngrediente`.
- No se debe llamar al backend.
- No se debe limpiar el formulario.

Ejemplos de mensajes esperados:

`El nombre no puede estar vacío`

`El stock debe ser un número`

`El stock no puede ser menor que 0`

Si los datos son válidos, el formulario debe continuar funcionando normalmente:

- Si no hay `ingredienteId`, se debe crear un ingrediente.
- Si existe `ingredienteId`, se debe actualizar el ingrediente.
- Se debe mostrar el mensaje correspondiente.
- Se debe limpiar el formulario.
- Se debe actualizar el listado de ingredientes.

5. Mostrar el detalle real de los pedidos (15 %)

Modificar el archivo `frontend/js/pedidos.js` para que el listado de pedidos muestre el detalle real de cada pedido.

Actualmente el listado puede mostrar solo los identificadores de los platos o un detalle incompleto. Se debe mostrar la información calculada que viene desde `GET /pedidos`.

Por cada pedido se debe mostrar:

- Identificador del pedido.
- Fecha del pedido.
- Detalle de los platos solicitados.
- Total general del pedido.

Por cada item del pedido se debe mostrar:

- Nombre del plato.
- Cantidad solicitada.
- Subtotal.

Formato visual mínimo esperado:

```
Pedido #1
Fecha: 2026-04-27T10:00:00.000Z
Items:
- Hamburguesa x 2 - Subtotal: $10000
- Cheeseburger x 1 - Subtotal: $4500
Total: $14500
```

Consideraciones:

- Se debe usar el arreglo `itemsDetalle` entregado por el backend.
- No se debe calcular el total nuevamente en el frontend.
- Si el pedido no tiene items, debe mostrarse un mensaje comprensible.

6. Agregar la pestaña Dashboard al menú y completar su funcionamiento (15 %)

Modificar las páginas del frontend para volver a incluir la pestaña **Dashboard** en el menú de navegación, que corresponde al archivo `dashboard.html`. El dashboard debe mostrar:

- Cantidad total de pedidos realizados.
- Platos realizados.
- Ingredientes consumidos.

Formato visual mínimo esperado:

Pedidos realizados: 2

Platos realizados:

Hamburguesa: 3

Cheeseburger: 1

Ingredientes consumidos:

Pan: 8

Carne: 4

Queso: 5

El apartado de ingredientes consumidos está incompleto en el backend.